

Semesteroppgave MUST1060

I dette prosjektet har jeg laget et program som tar midi inn fra et instrument og endrer på intonasjonen for å renere klanger. Målet med programmet var å ha et verktøy som gjør det mulig å høre forskjellen mellom vanlig «pianostemming» og rene intervaller i akkorder. Rapporten tar for seg ideen bak prosjektet, arbeidsprosessen, den tekniske løsningen og en evaluering av resultatet.

Ideen

Ideen til prosjektet kom fra min interesse for musikkteori og kormusikk. Tanken for å lage et program der man faktisk får hørt hvordan rene akkorder faktisk høres ut, noe man ikke har mulighet til med et vanlig instrument. Ideen er kanskje mer et verktøy for gehørtraining og forståelse av korintonasjon, men jeg ser det som anvendelig å bruke det til å lage musikk som klinger rent.

Prosessen

Arbeidsprosessen startet med å manuelt stemme en akkord ved hjelp av cent funksjonaliteten i Csound. Dette var en test av konseptet før jeg bygde videre på programmet. Neste steg var å lage et system som gjorde dette automatisk for hver akkord som ble spilt. Da lagde jeg en tabell som inneholdt alle cent-justeringene for de 12 intervallene. Deretter lagret jeg alle aktive toner i en global tilstand og brukte den til å finne den laveste tonen og gjøre den til grunntone. Dette fungerte greit, men hvis man spilte en akkord i annen omvendning ville grunntonen være feil. Derfor lagde jeg en ny *opcode* som fant den eldste tonen av alle tonene som spilles. Dette gjorde at man kan velge grunntone ved å spille den først.

Etterhvert som jeg fikk til å stemme akkorder automatisk lagde jeg funksjonalitet til å velge bølgeformer dette er interessant for å se hvordan rene klanger høres ut med forskjellige klangfarger. Istedenfor å bruke innebygde *saw-* og *square-wave* funksjoner konstruerte jeg disse av sinusbølger for å håpe at dette ville klinge best mulig når jeg la til stemming basert på brøker. Til slutt lagde en slags *vocoder* som brukte mikrofonen og et bæresignal. Her ble det litt klipp og lim fra internett, og det fungerer sånn så som så.

Noe av det siste jeg la til var muligheten til å bytte mellom vanlig stemming og ren stemming. Dette var veldig fint å ha da jeg tiltenkte dette som et verktøy for trening på lytting og ved å kunne sammenligne på dette viset får man mer ut av programmet. Tidligere versjon hadde kun ren stemming. Det aller siste som ble lagt til var stemming basert på brøker istedenfor cent verdiene, når man kjører programmet ser man at cent-verdiene er veldig nære brøkene, men litt annerledes, det er vanskelig å høre forskjell, men kan være god trening å lytte etter små forskjeller.

Teknisk løsning

Jeg føler den tekniske løsningen er ganske ryddig, bruk av egne *opcode*-er gjør programmet enklere å lese samt beskrivende kommentarer hjelper en som er ukjent med programmet å forstå hva som skjer.

Jeg lagde også en *opcode* som heter *PrintHeldNotesTable* som skriver ut alle aktive toner i konsollen. Den oppdateres hver gang det skjer en endring i hvilke toner som spilles for øyeblikket. Dette var et viktig verktøy i feilsøkingen av programmet, men brukes også som et «grensesnitt» ved bruk for å se hva som faktisk foregår i programmet. Jeg forsøkte å få den til å oppdatere tabellen hvert gang man skrudde på en *midi-knob* eller lignende, men fikk aldri dette helt til å fungere.

Evaluering og refleksjon

Koden fungerer nesten som tiltenkt, de eneste tingene jeg ikke er så fornøyd med er at tabellen ikke oppdateres hele tiden og at *vocoderen* ikke høres så bra ut. Det var dog å få stemmingen til å fungere som var det viktigste for meg da dette var noe «nytt», å få en *vocoder* til å funke er trivielt i den form at det finnes mange maler og eksempler på hvordan man gjør dette.

Jeg ønsket også å få til å bytte grunntone ved å skru på en «knob», dette fikk jeg til å fungere, MEN jeg klarte ikke å oppdatere tabellen hver gang og dermed var den vanskelig å bruke da hvis man endret toner ville programmet velge ny grunntone og justeringen ble ikke lagret. Dette hadde jeg jobbet mer med om jeg skulle videreutviklet prosjektet.

Med tanke på kodekvalitet føler jeg at jeg har fått det godt til, jeg har brukt eksempelfilen som mal og lagt inn rikelig med kommentarer som gjør koden enkel å navigere. Noe jeg har gjort som ikke var i malen var å benytte indentering, dette gjør jeg av vane fra tidligere programmering, og det gjør at jeg kan «folde» koden i editor slik at den blir enkel å jobbe med. I tillegg liker jeg å bruke engelsk når jeg koder da det overlapper bedre med kodespråket.

Jeg har kanskje ikke demonstrert ulike aspekter av effekter og instrument som er musikalsk interessante. Dette ble rett og slett litt glemt, men jeg føler allikevel at prosjektet har en tydelig musikalsk idé som er interessant i seg selv. Ved opplastingen ligger det to filer, dette er *Heyr Himna Smidur* som er stemt ved å sende MIDI-noter til programmet og la den spille av. Den første er midi-filen er *streit* så det er litt tilfeldig hvilken tone som blir grunntonen, ofte den laveste i akkorden, og den nr 2 er justert slik at grunntonen kommer litt før resten av akkorden, det er ikke perfekt, men det er tydelige forskjeller i visse akkorder som er interessant. Man kan argumentere for at lav 7-er burde være justert annerledes i denne moll-sangen, men jeg fulgte bare tabellen min slavisk. Man hører også at det ikke tas hensyn til at melodien burde være uendret, slik at den høres ren ut. Det går an å fikse, men det er allikevel interessant å høre eksemplene. Jeg valgte *saw-wave* for å tydeliggjøre forskjellene.